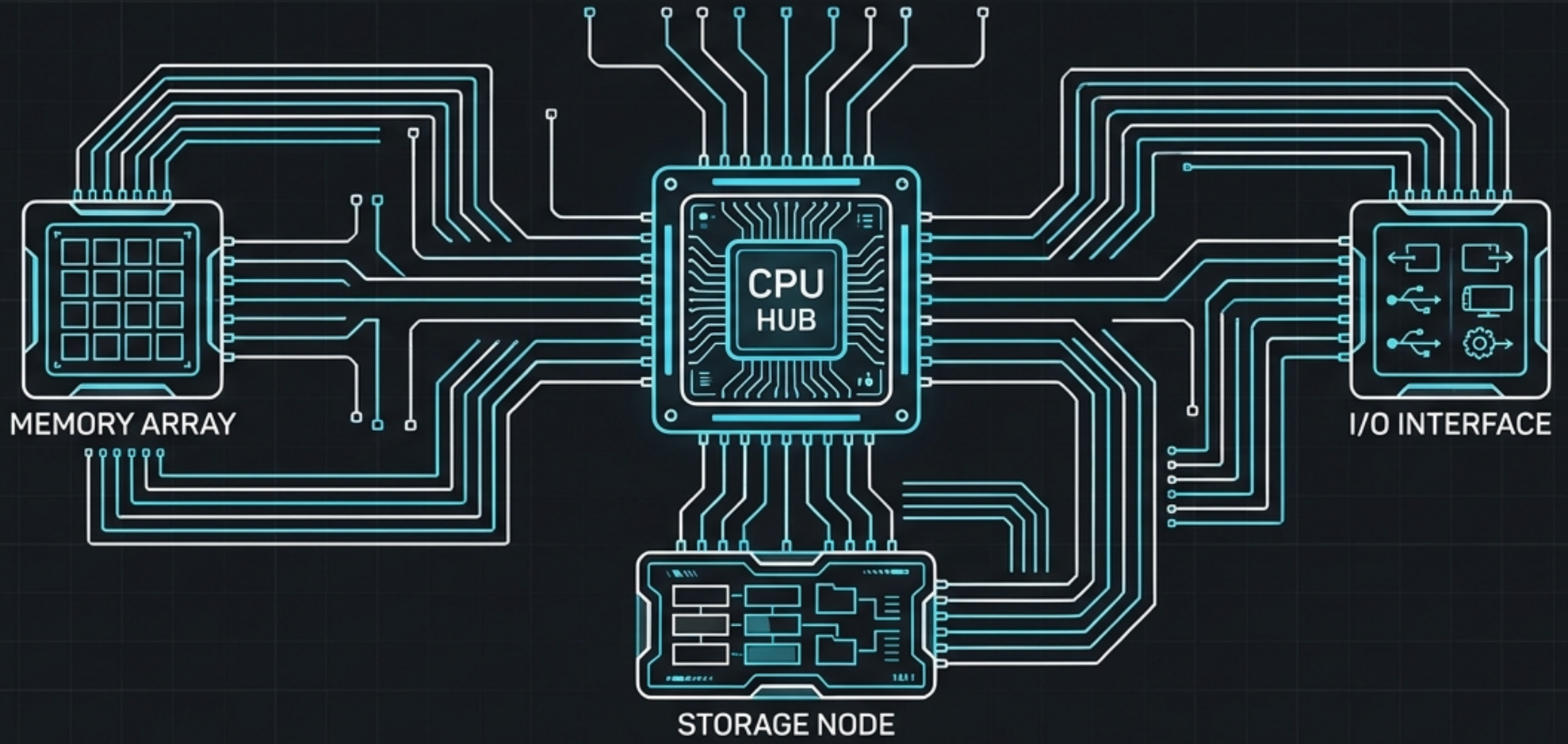
BCL1213 Operating Systems Mastery Guide

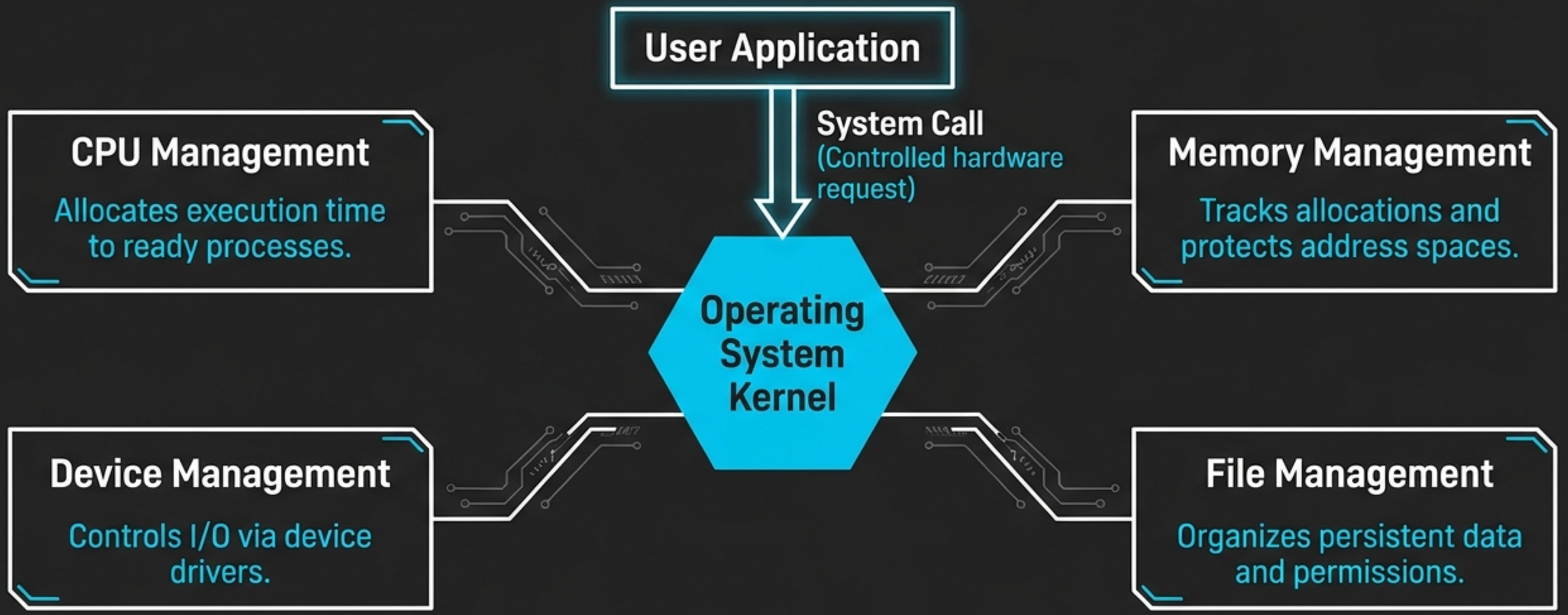
Comprehensive Exam Revision (Chapters 1–4)



Self-Study Visual Deck | Chapters 1–4 Focus | Fully Worked Examples | Verified Technical Data

Gemini Notebook

The OS as the Central Building Manager



Key Insight: The OS acts as an intermediary. Applications cannot touch hardw directly; they must request services from the central manager.

Gemini Notebook

Evolution of System Architectures

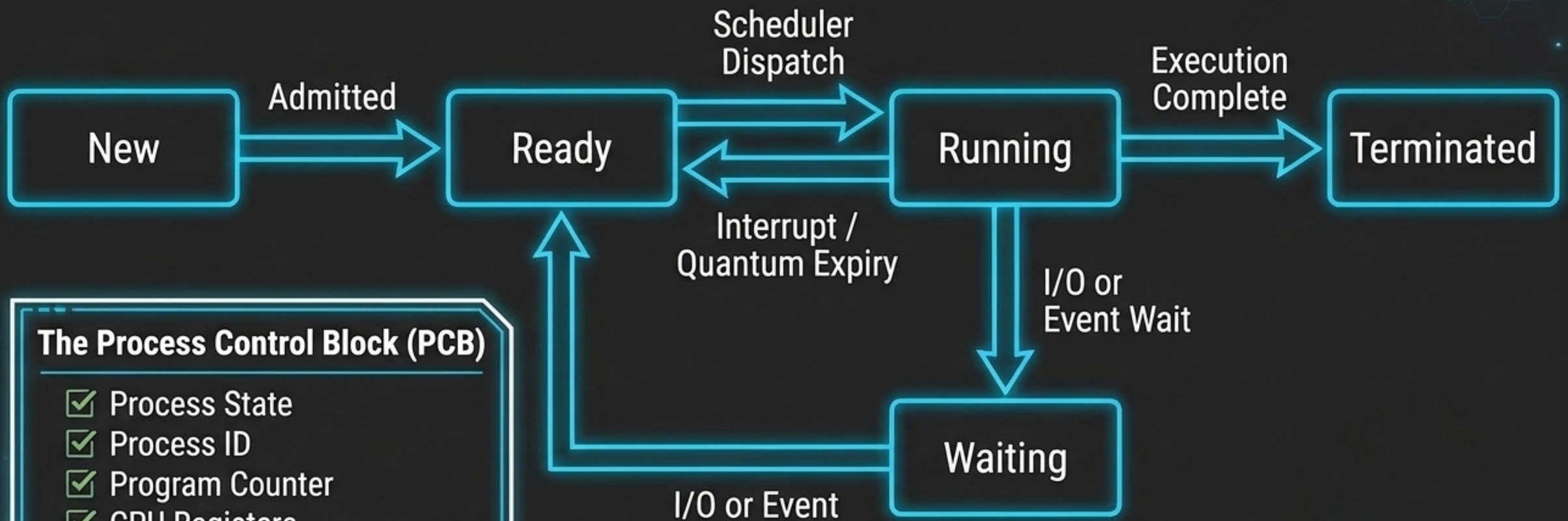
Architecture	Core Concept	Use Case & Limitation
Batch	Executes submitted jobs sequentially.	Payroll/offline jobs. Poor for urgent interactive control.
Multiprogramming	Keeps multiple jobs in RAM. CPU switches during I/O waits.	Improves CPU utilization. No simultaneous execution.
Time-sharing	Rapidly switches tasks via time-slicing.	Responsive multiuser work. Overhead costs.
Real-time	Strict timing and functional requirements.	Hard deadlines are mandatory.
Distributed	Autonomous nodes coordinate as integrated environment.	Each node has own CPU/memory. Network dependency.

Crucial Distinction: Utilization vs. Interactivity

Recall Question: Why does time-sharing not guarantee hard real-time deadlines? (Fast average response ≠ guaranteed deadline).

Gemini Notebook

The Architecture of Process Execution



The Process Control Block (PCB)

- ☑ Process State
- ☑ Process ID
- ☑ Program Counter
- ☑ CPU Registers

Used during Context Switching to save/load execution states.

Gemini Notebook

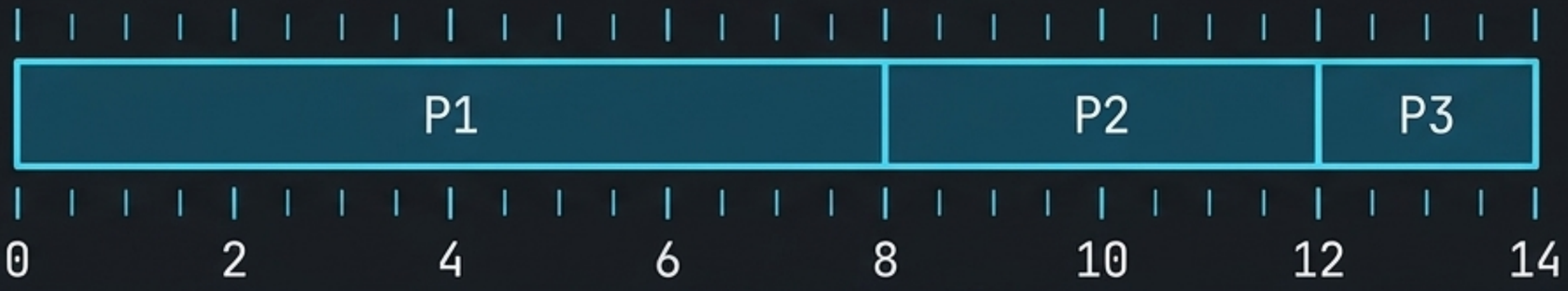
First-Come, First-Served (FCFS) Scheduling

Top Panel (Formulas)

Assumptions: 1 CPU, 0 context-switch cost.
Turnaround Time (TAT) = Completion (CT) - Arrival (AT)
Waiting Time (WT) = TAT - Burst (BT)

Execution Trace:

Dataset:
P1: AT=0, BT=8
P2: AT=1, BT=4
P3: AT=2, BT=2



Metrics Matrix:

P1: TAT = 8ms, WT = 0ms	
P2: TAT = 11ms, WT = 7ms	
P3: TAT = 12ms, WT = 10ms	

System Averages:
Turnaround = 10.33 ms
Waiting = 5.67 ms

Gemini Notebook

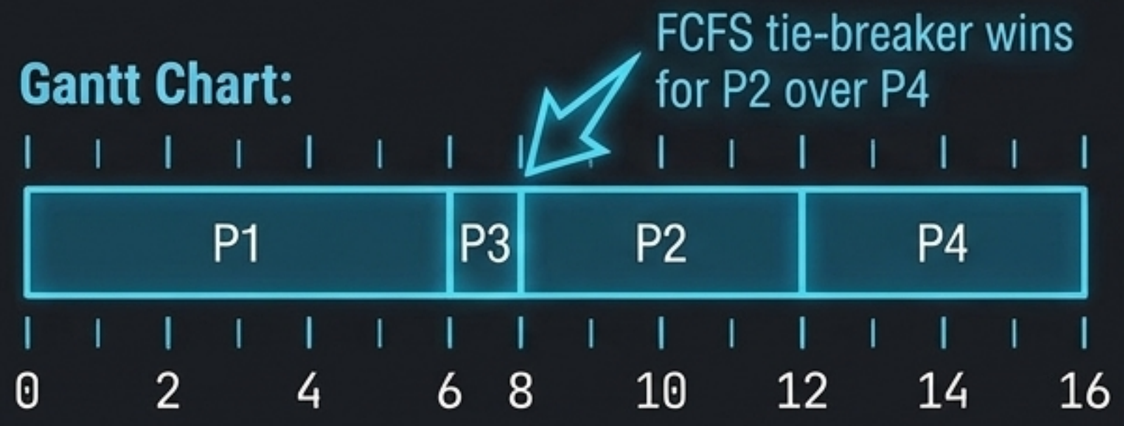
Non-Preemptive Routing: Shortest Job First vs. Priority

Shortest Job First (SJF)

Rule: Select shortest available burst.
Tie-breaker = FCFS arrival time.

Dataset: P1(0,7), P2(2,4), P3(4,1), P4(5,4)

Gantt Chart:



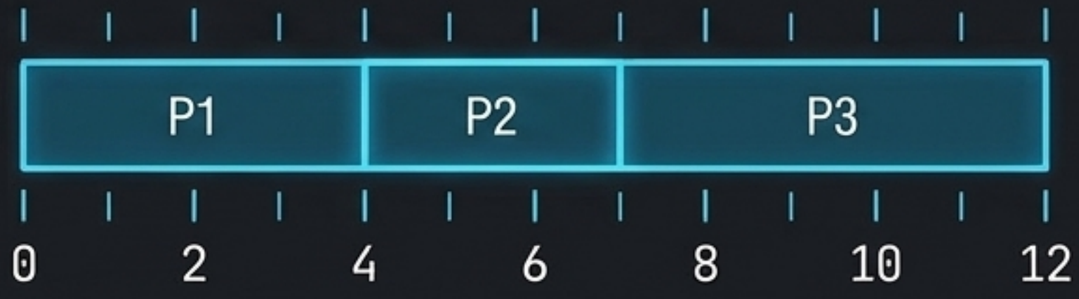
Averages: TAT = 8.00ms | WT = 4.00ms

Priority Scheduling

Rule: Lower integer = Higher Priority.

Dataset:
P1(0,4, Pri:2), P2(1,3, Pri:1), P3(2,5, Pri:3)

Gantt Chart:



Averages: TAT = 6.67ms | WT = 2.67ms

Calculation Error Risk: Always remember to subtract non-zero arrival times when calculating Turnaround!

Gemini Notebook

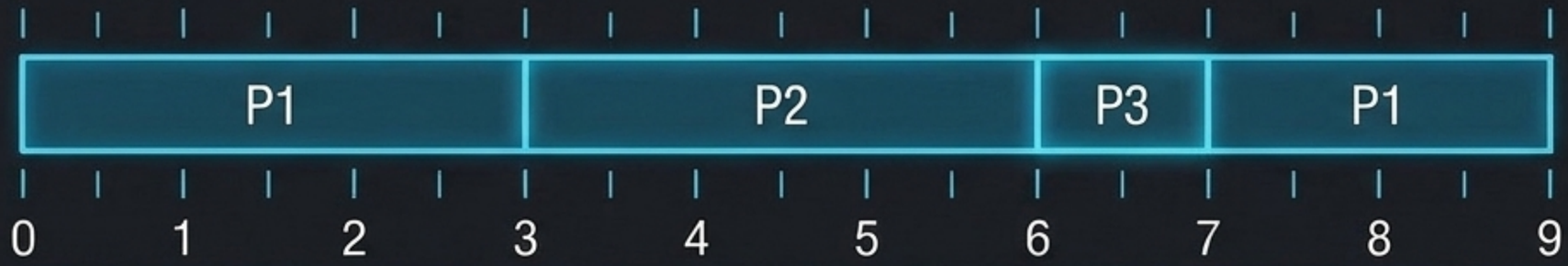
Preemptive Scheduling: Round Robin (RR)

Rule: Time Quantum (Q) = 3ms. Dataset: P1(AT:0, BT:5), P2(AT:1, BT:3), P3(AT:2, BT:1)

Queue Tracking Timeline

t=0: Queue [ ]. P1 runs.
t=3: P1 preempted (2ms left). Queue is [P2, P3, P1]. P2 runs.
t=6: P2 completes. Queue is [P3, P1]. P3 runs.
t=7: P3 completes. Queue is [P1]. P1 runs.
t=9: P1 completes. Queue is [ ].

Gantt Chart:

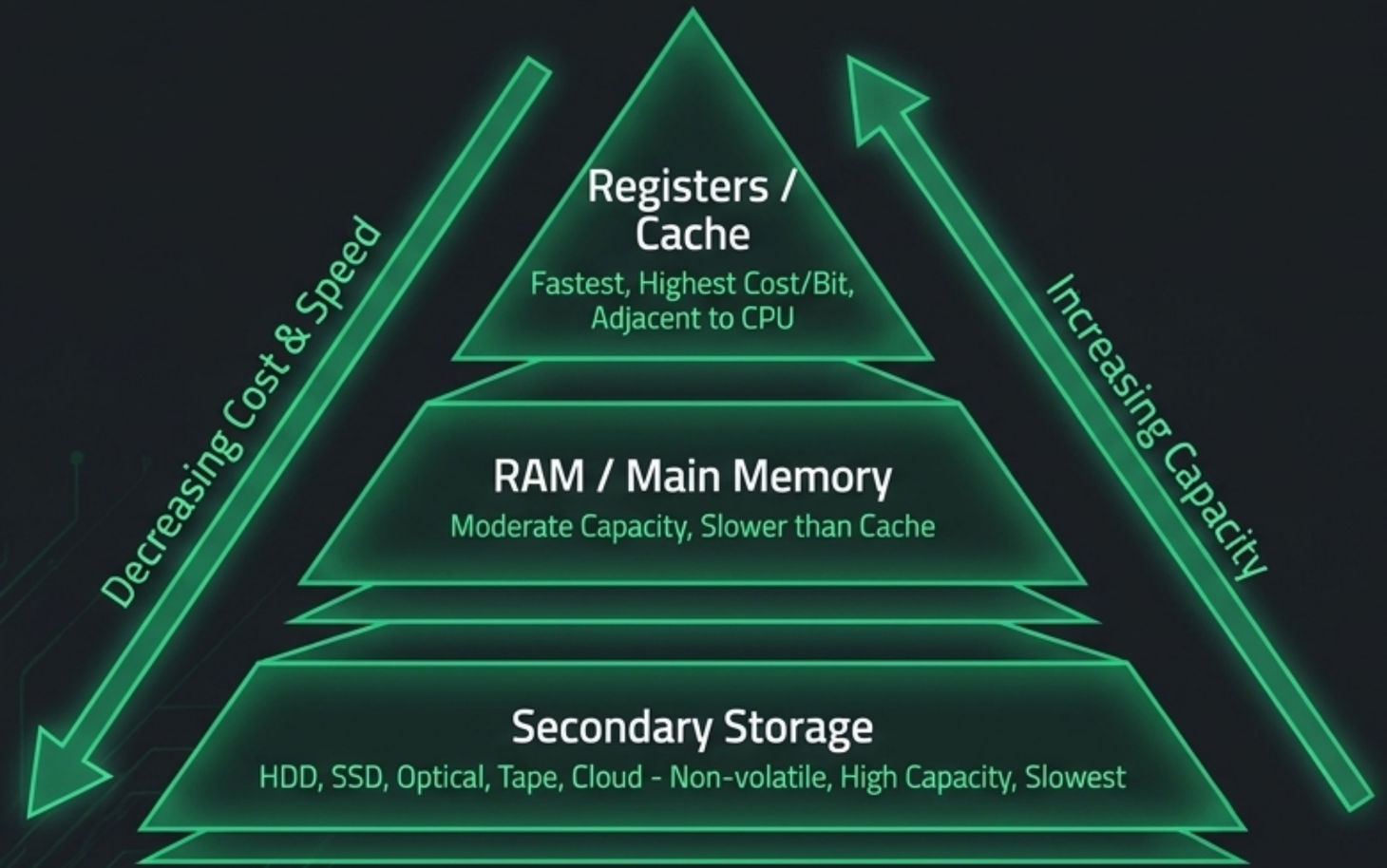


System Averages:
TAT = 6.33ms
WT = 3.33ms

Key Insight: Quantum too small = extreme context-switch overhead.
Quantum too large = degenerates into standard FCFS.

Gemini Notebook

The Memory Hardware Hierarchy



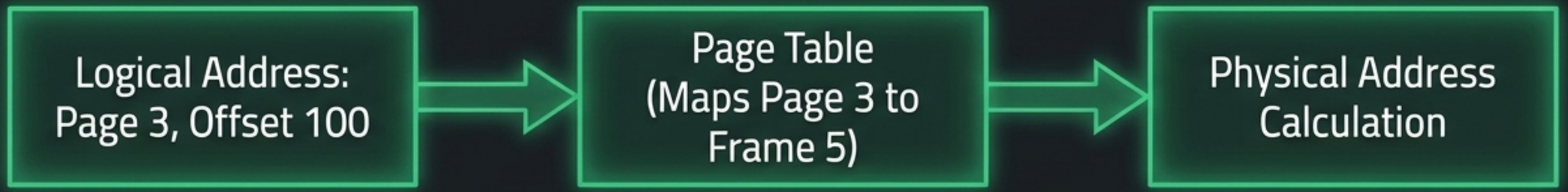
Virtual Memory Mechanics

How does a process needing 12 GB run on 8 GB of physical RAM?

Demand Paging. The OS maps logical addresses to physical memory and swaps inactive pages to backing storage (avoids thrashing if limits are respected).

Gemini Notebook

Logical to Physical Address Translation



Math Execution:

Scenario Parameters:

- Page Size = 4 KiB (4096 bytes)
- Formula: Phys Addr = (Frame × Page Size) + Offset

Calculation:

1. Validity Check: Is $0 \leq 100 < 4096$? YES.
2. Calculation: $(5 \times 4096) + 100$
3. Result: 20,480 + 100 = 20,580 bytes.

Critical Rule: The offset must fit inside the page size. An offset of 100 KB on a 4 KB page is mathematically invalid. Validate first!

Gemini Notebook

Fragmentation & Dynamic Memory Allocation

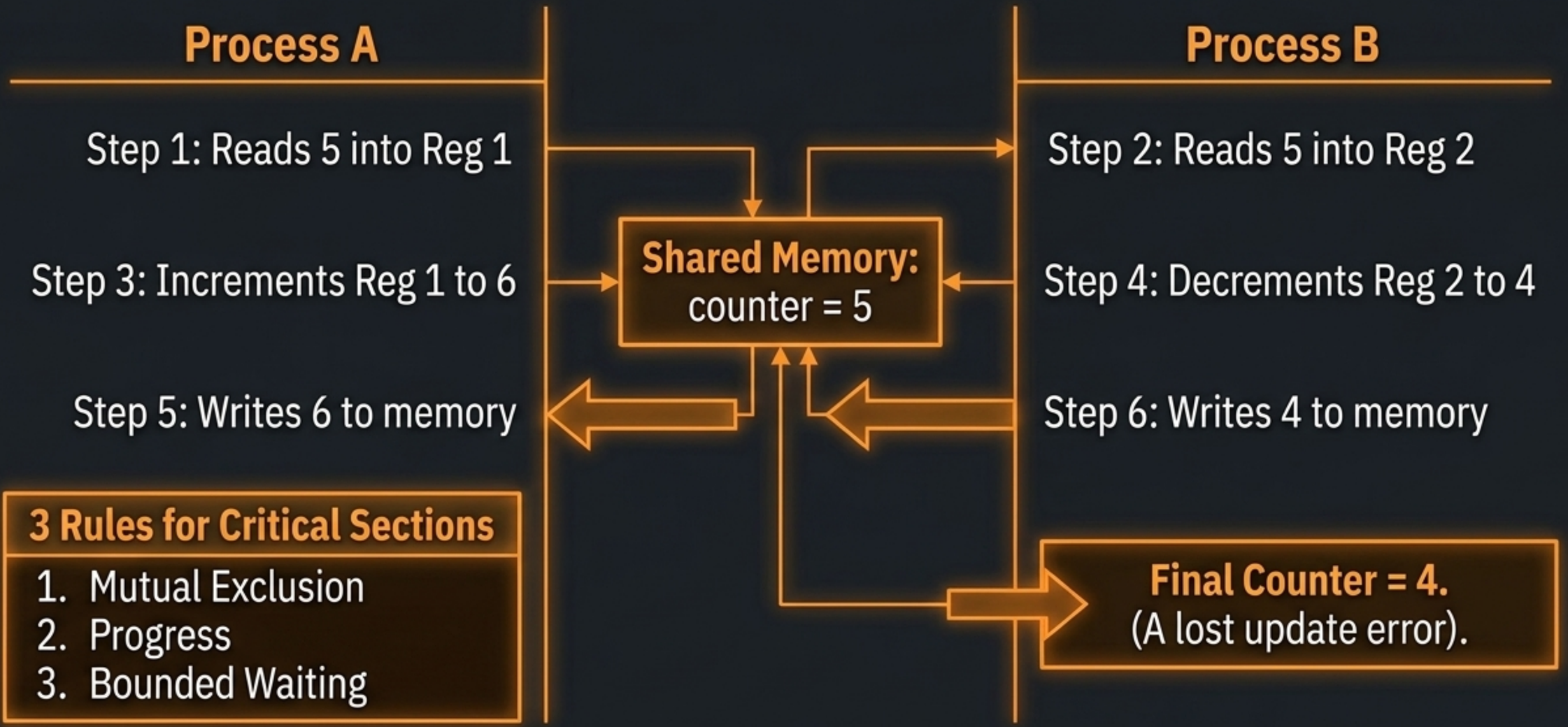
Internal Fragmentation	External Fragmentation
Unused space trapped inside a fixed block (Paging / Fixed Partitions).	Free space is scattered in non-contiguous gaps (Dynamic Allocation). Eliminated by paging.

Memory Holes Available: [100] [500] [200] [300] [600] KiB.
Request: Allocate 212 KiB.



Gemini Notebook

The Critical Section & Race Conditions



3 Rules for Critical Sections

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Gemini Notebook

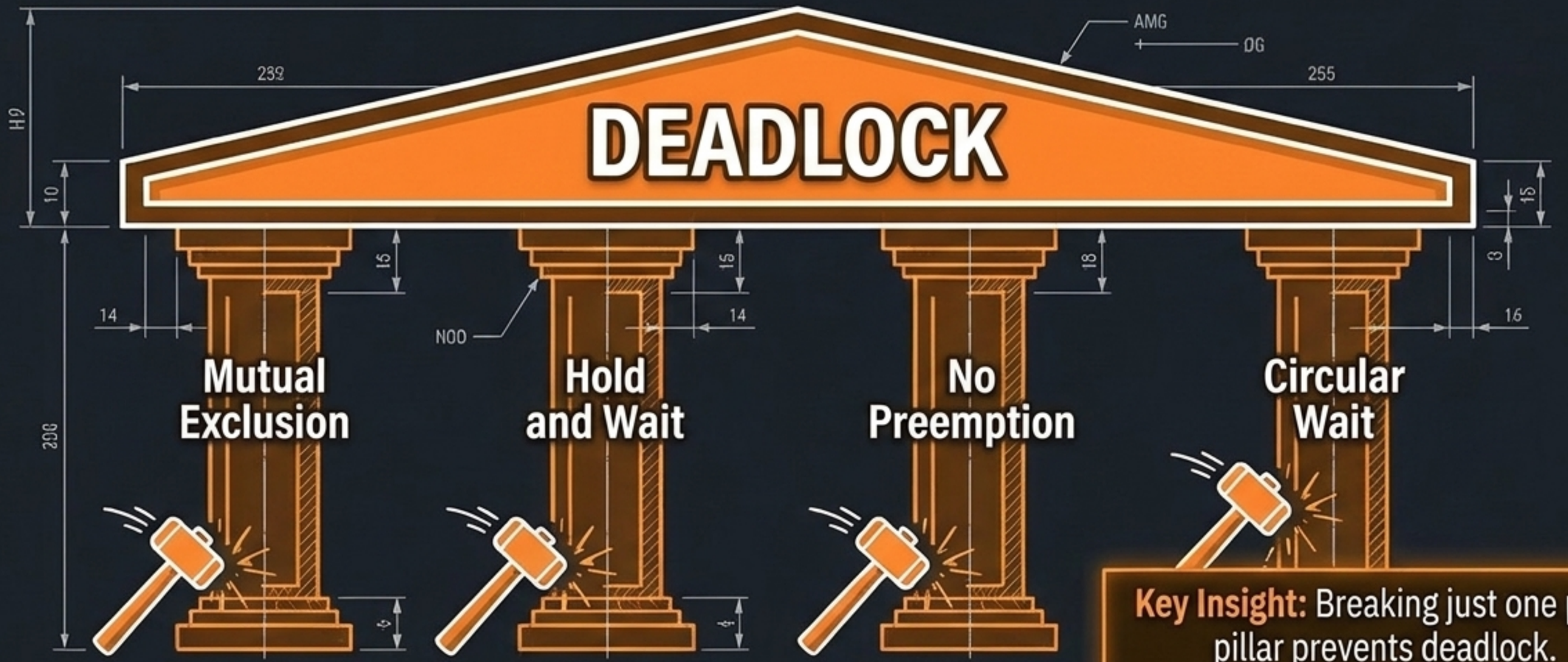
Process Synchronization Primitives

Mutex Lock	Binary Semaphore	Counting Semaphore	Condition Variable
Strict ownership (only acquiring thread can release). Protects single critical sections. Operations: acquire/release.	Restricted to 0 or 1. No ownership (any thread can signal). Used for binary mutual exclusion or signaling. Operations: wait/signal.	A count of available permits. No owner. Manages access to finite pools of identical resource instances.	Blocks until a condition is signaled. Requires an associated Mutex. Operations: wait, signal, broadcast.

Recall Question: Why must a condition variable be checked in a loop? (To protect against spurious wakeups or state changes by other threads before re-acquiring the mutex).

Gemini Notebook

Deadlocks: The Coffman Conditions



Gemini Notebook

Diagnostic Check: Common Pitfalls & Explanations

Q: How does paging eliminate external fragmentation?

A: Any logical page fits into any physical frame; no contiguous gaps are wasted.

Q: Why check a segmentation offset before adding the base?

A: A limit check ensures no out-of-bounds memory access, preventing illegal access to other programs.

Q: What is the difference between waiting for CPU vs. I/O?

A: CPU wait places the process in the Ready queue (preempted); I/O wait places it in the Blocked queue.

Q: Are distributed nodes controlled by one central CPU?

A: No. Each autonomous node in a distributed system has its own distinct processor and memory.

Gemini Notebook

Final Revision Checklist & Formulas

Concept Checklist

- [ ] Contrast time-sharing vs. multiprogramming.
- [ ] Trace the 5-state process flowchart.
- [ ] Recalculate FCFS, SJF, Priority, and RR scheduling.
- [ ] Execute valid physical address translations.
- [ ] Trace race conditions and define Mutex/Semaphore differences.
- [ ] Name the 4 Coffman deadlock conditions.

Formula Cheat Sheet

TAT = CT - Arrival Time

WT = TAT - Burst Time

Phys Addr = (Frame × Page Size) + Offset

Validity Check: 0 ≤ Offset < Page Size

You are ready. Master the mechanics, validate the variables.

Gemini Notebook